MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

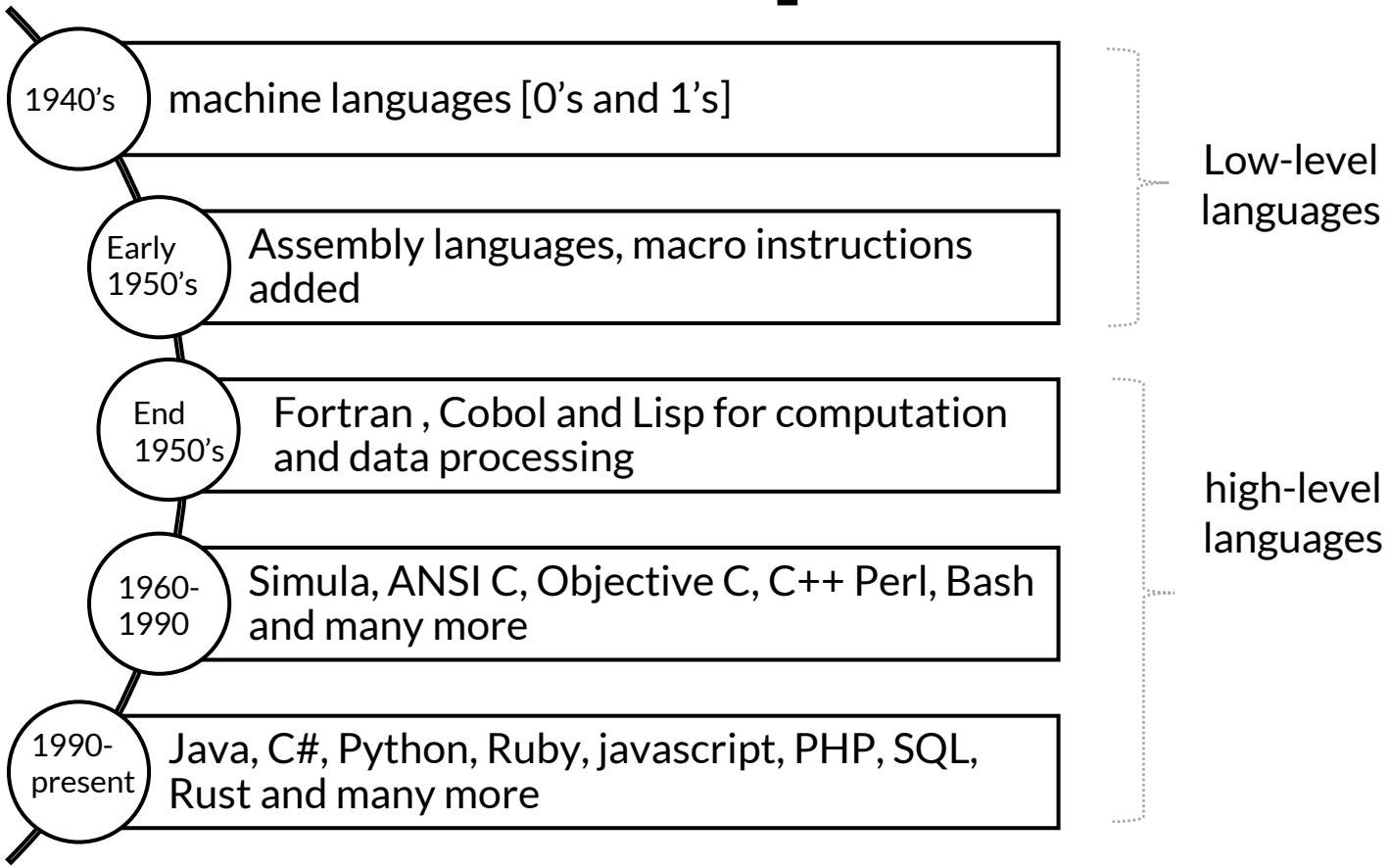
Compilers

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning Outcomes

- Understand what a compiler is
- Describe how compilers work
- Explain the difference between assemblers and compilers

Why we need a compiler

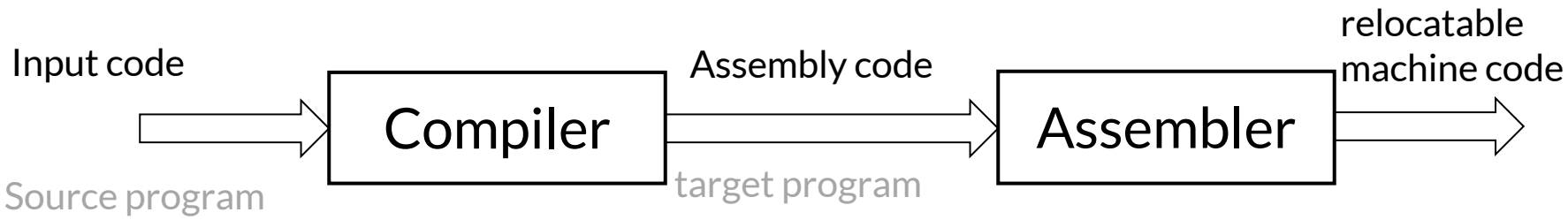


- Compilers promote the use of high-level languages by minimizing the execution overhead of the programs written in these languages.
- Compilers are also critical in making high-performance computer architectures

What is a compiler

- *"A compiler is a program that can read a program in one language - the source language - and translate it into an equivalent program in another language - the target language"
- The main aspect of a compiler is:

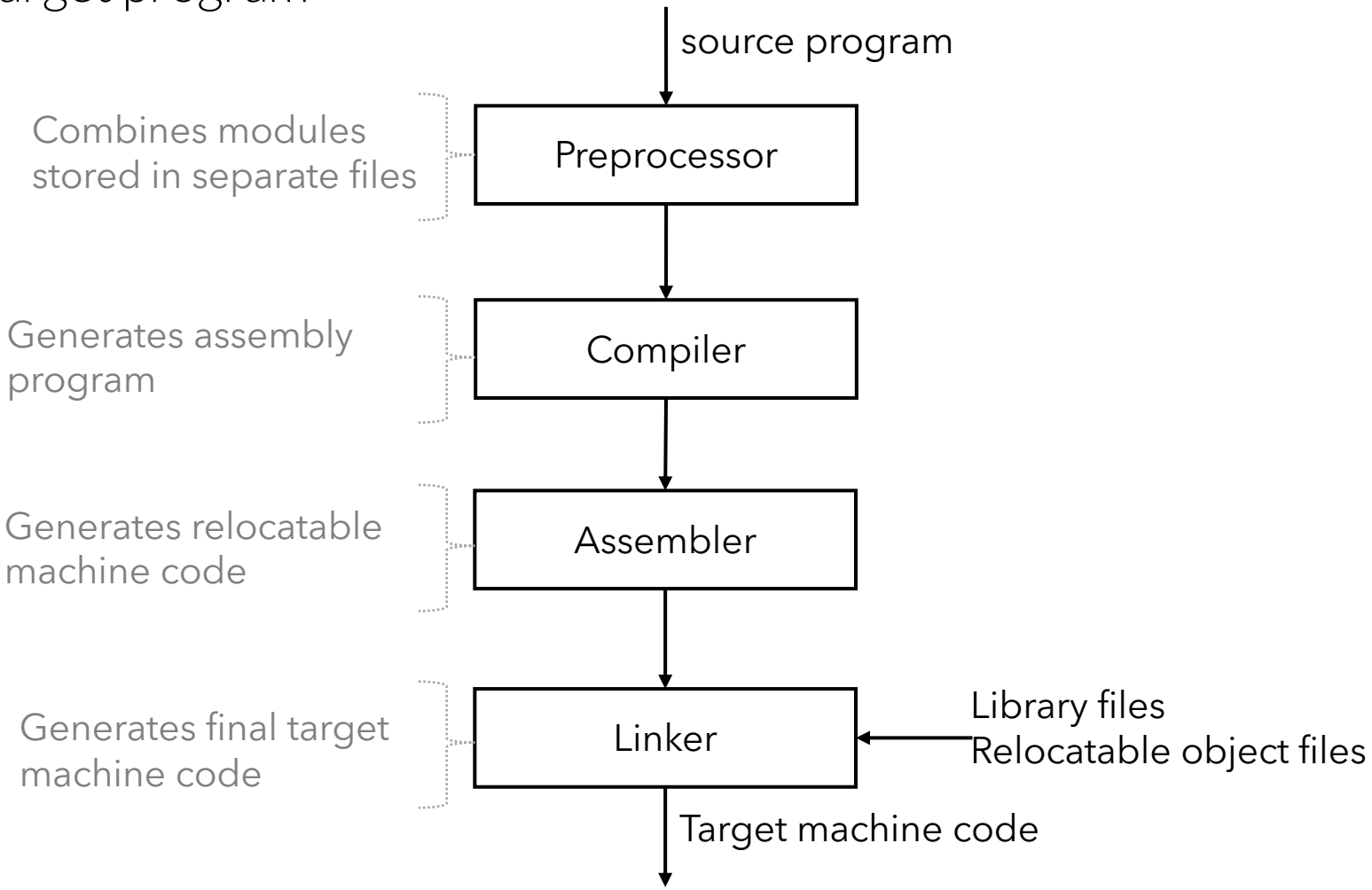
  "The meaning (semantics) of input program must be preserved"



*Referenced book: Aho *et al.* "Compilers: Principles, Techniques and Tools" 2nd edition
https://dl.acm.org/citation.cfm?id=1177220

A basic language processing system

- In addition to a compiler, several other programs may be required to create an executable target program

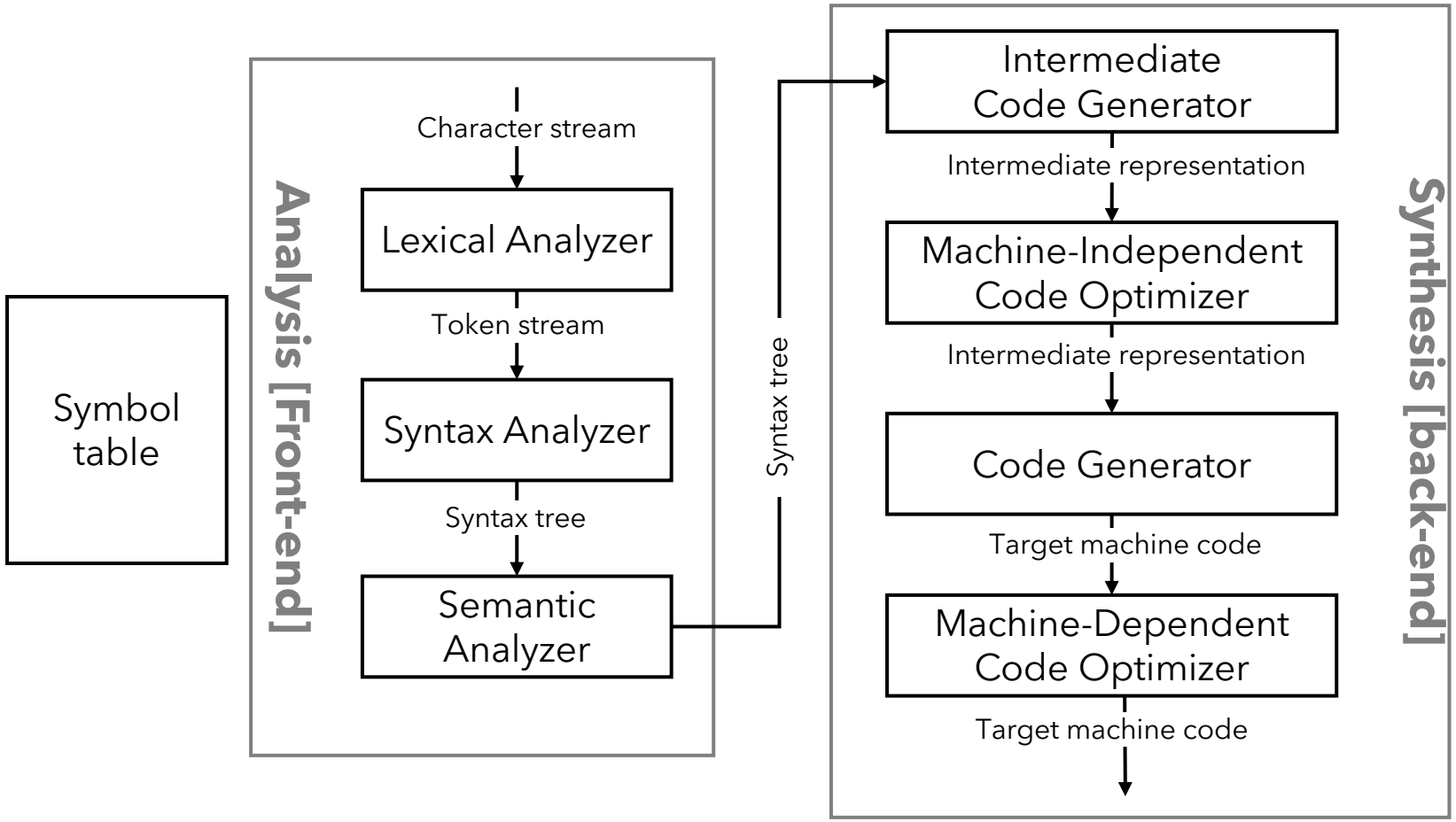


Purpose of a Compiler

- **Primary:**
  - To comply with the specifications of the input language
  - To generate correct code for the target machine
- **Secondary:**
  - To generate a fast output code with low memory footprint (reliable optimisations)
  - To have a low compilation time (must be linearly proportional to the size of input code)
  - To provide good diagnostics (debugging/ error reporting)

Structure of a basic Compiler

- Compiler has two main parts:
  - **Analysis (Front-end)**
  - **Synthesis (Back-end)**



Front-end phases

- **Lexical Analysis:** Reads the input characters of the source program, group them into lexemes, and produce as output a sequence of tokens of the form <token-name, attribute-value> for each lexeme

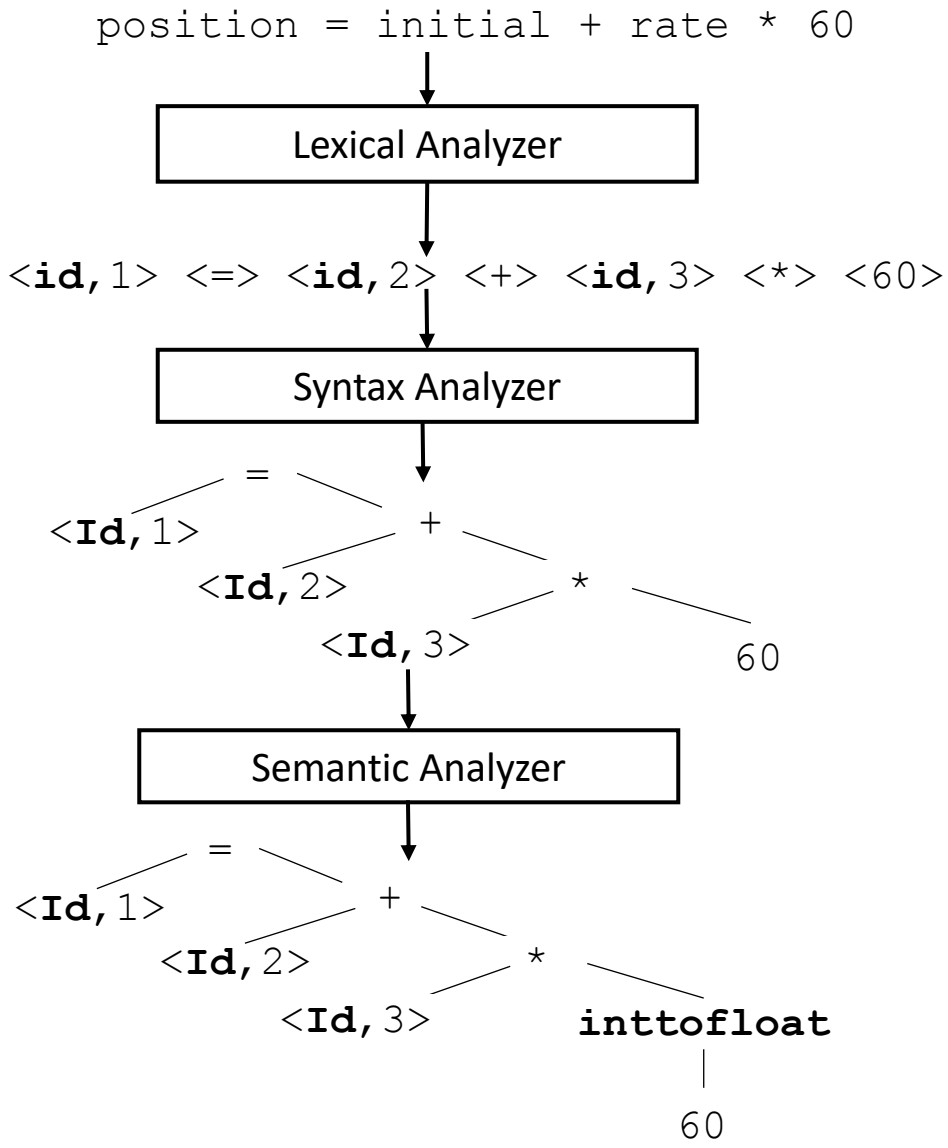
- **Syntax Analysis:** Uses tokens, produced by the lexical analyser, to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.

- **Semantic analysis:** Uses the syntax tree and the information in the symbol table to check the source program structure for semantic consistency with the language definition

Example code: assignment

1	position	.....
2	initial	.....
3	rate	.....

SYMBOL TABLE



Back-end phases

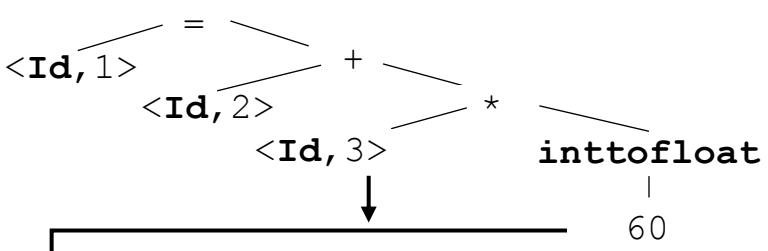
- **Intermediate Code Generation:** Generates an explicit low-level or machine-like intermediate representation (for an abstract machine). Two important properties:
  - It should be easy to produce.
  - It should be easy to translate into the target machine.

- **Code Optimization:** This phase attempts to improve the machine-independent intermediate code so that better (faster, concise, power efficient, etc.) code can be generated for any target machine.

- **Code Generation:** The code generator takes as input an intermediate representation of the source program and maps it into the target language.

Example code

Syntax tree generated by the Front-end



Intermediate Code Generator

```
t1  = intofloat(60)
t2  = id3 * t1
t3  = id2 + t2
Id1 = t3
```

Code optimizer

```
t1  = id3 * 60.0
t2  = id2 + t1
```

Code Generator

```
flw      f2, id3
li       t0, 60.0
fcvt.s.w f3, t0
fmul.s   f2, f2, f3
flw      f1, id2
fadd.s   f1, f1, f2
fsw      f1, id1, f3
```

1	position	.....
2	initial	.....
3	rate	.....

SYMBOL TABLE

Compiler vs Assemblers

	COMPILER	ASSEMBLER
Basic functionality	Generates the assembly language code	Generates the (relocatable) machine code.
Input	Source code (typically in a high-level language)	Assembly language code (low-level language)
Passes/sub-processes	The compilation phases are lexical analyzer, syntax analyzer, semantic analyzer, intermediate code generation, code optimization, code generation.	Assembler makes fewer and simpler passes over the given input without any optimization.
Output	The assembly code in a mnemonic version of machine code.	The machine code in a relocatable binary representation.

Applications of Compiler Technology

- Implementation of high-Level programming languages
  - The programmer expresses an algorithm using a high-level language, and the compiler must translate that program to the target language
- Optimizations for Computer Architectures
  - Parallelism: To execute code in less time
  - Memory: To generate minimal size code and better utilize memory
- Design of new computer architectures
  - The performance of a computer system is determined not by its clock speed but also by how well compilers can exploit its features.
- Binary Translation
  - Translating binary code for one machine to that of another, allowing a machine to run programs originally compiled for another instruction set.
- Hardware-synthesis
  - Transforming hardware designs, described in hardware description languages, automatically into gate-level design. The generated designs are then mapped to transistors and eventually to a physical layout.

Summary

Compilation is similar to assembly except:

- High level languages are richer – higher functionality described in fewer expressions
- Semantic analysis is more complex
- High level language constructs such as switch, arrays, tables etc have to be expressed in assembly language
- Code generation is more complex

Compilers normally provide optimisation

- generation of code that does the same thing in fewer instructions
- many techniques: loop unrolling, instruction reordering, optimisation of registers and memory…

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Instruction Set Architectures

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning Outcomes

- Understand and explain different classes of ISAs
- Understand and explain RISC and CISC architectures
- Understand latest trends in computer architecture

Classifying ISAs

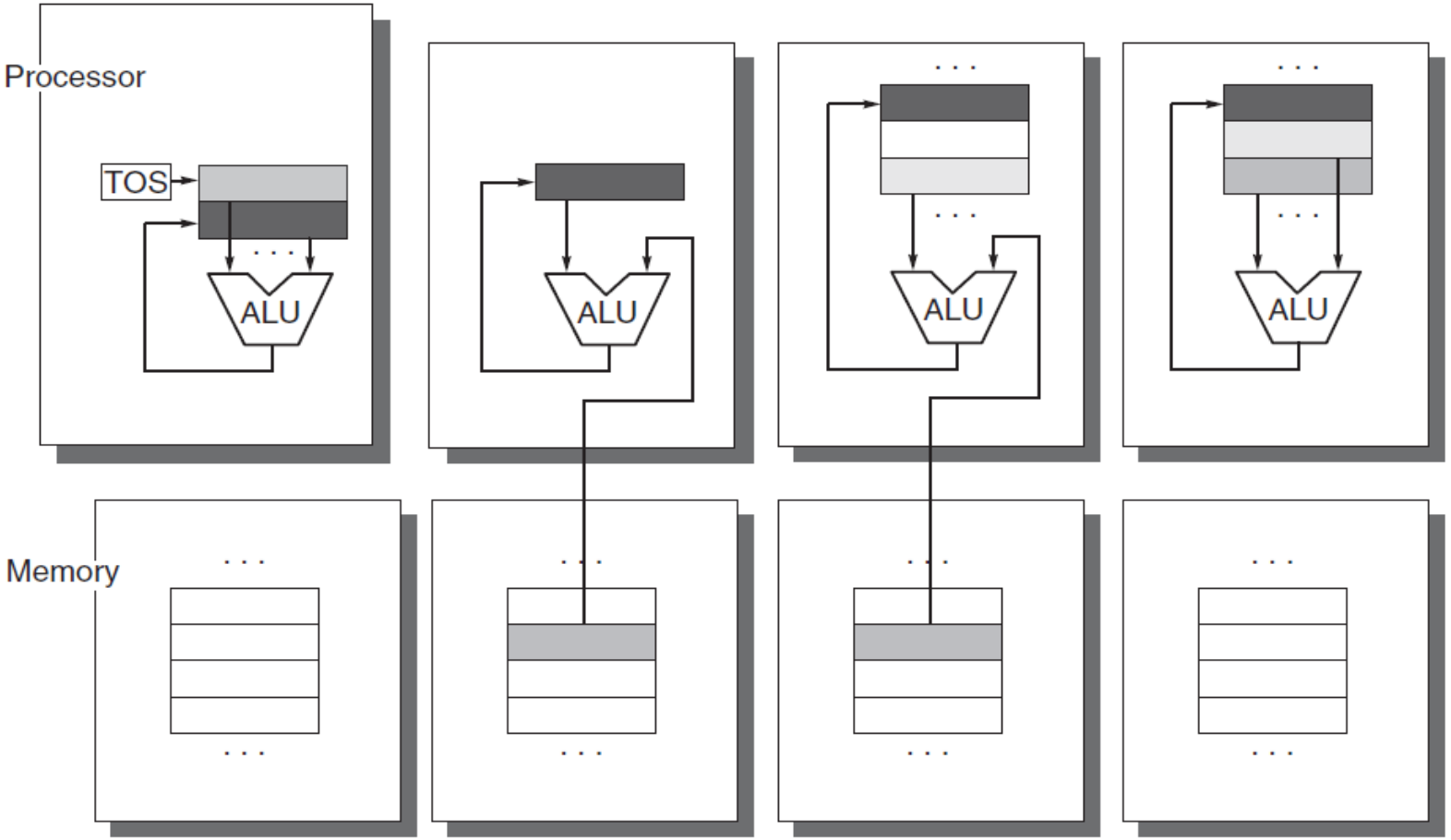
- Typically, instruction set architectures (ISAs) are classified based on the type of internal storage in a processor. This internal storage could be a stack, an accumulator, or a set of registers.

- Operands may be named explicitly or implicitly:
  - The operands in a stack architecture are defined implicitly on the top of the stack
  - In an accumulator architecture one operand is implicitly the accumulator.
  - The general-purpose register architectures have only explicit operands—either registers or memory locations

Referenced book: John L. Hennessy and David A. Patterson “Computer Architecture A Quantitative Approach” (5th edition)

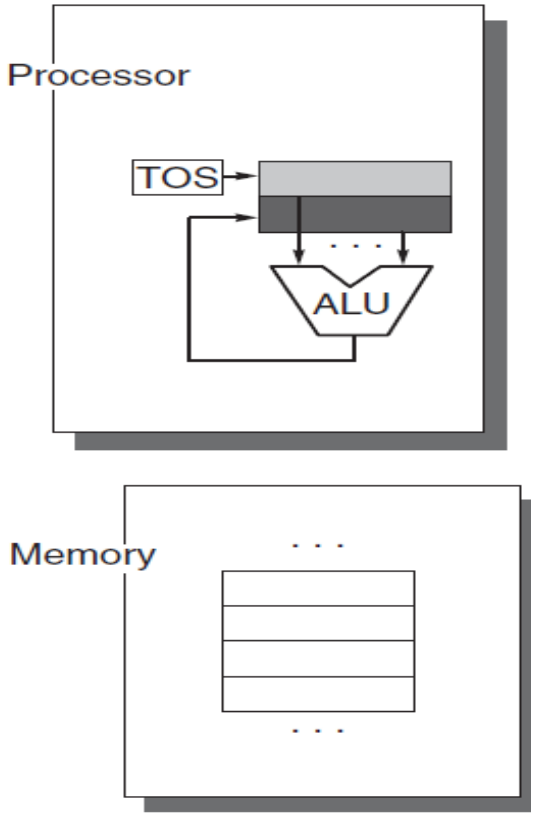
Classifying ISAs

This figure illustrated the operand locations for four instruction set architecture classes



Stack Based ISA

The figure shows how the code sequence C = A + B would typically appear for a stack based ISA

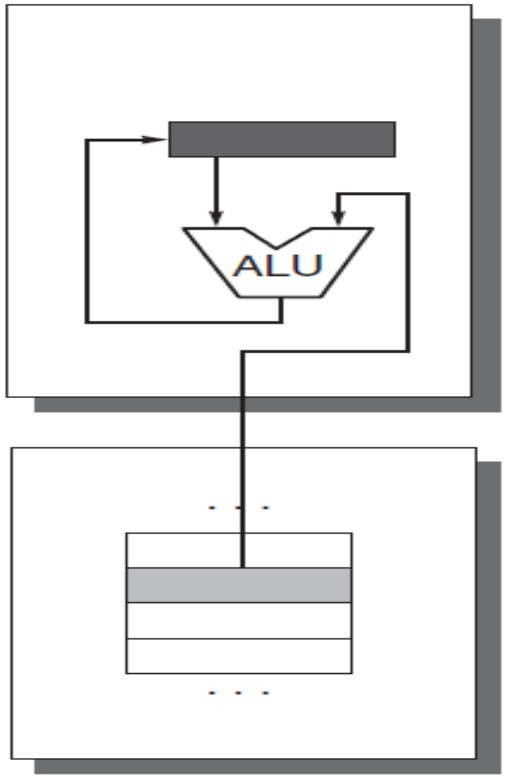


(a) Stack

Stack
Push A
Push B
Add
Pop C

Accumulator Based ISA

The figure shows how the code sequence C = A + B would typically appear for an accumulator based ISA

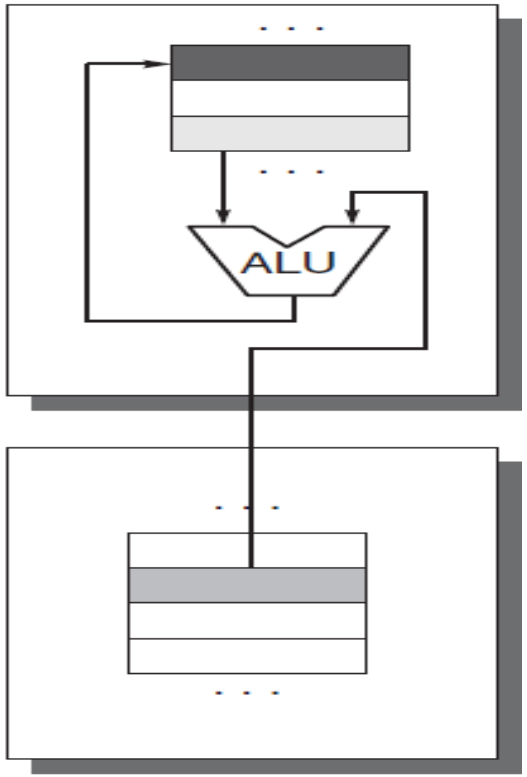


(b) Accumulator

Accumulator
Load A
Add B
Store C

Register-Memory Based ISA

The figure shows how the code sequence C = A + B would typically appear for a register(register-memory) based ISA based ISA

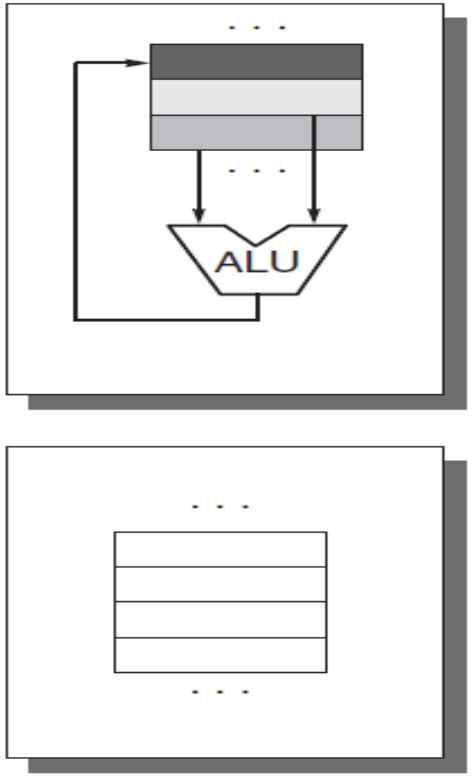


(c) Register-memory

Register-memory
Load R1, A
Add R3, R1, B
Store R3, C

Load-Store Based ISA

The figure shows how the code sequence C = A + B would typically appear for a register(load-store) based ISA based ISA



(d) Register-register/ load-store

Load-store
Load R1, A
Load R2, B
Add R3, R1, R2
Store R3, C

Classifying ISAs

Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1,A	Load R1,A
Push B	Add B	Add R3,R1,B	Load R2,B
Add	Store C	Store R3,C	Add R3,R1,R2
Pop C			Store R3,C

Classifying ISAs

The table shows typical combinations of memory operands and total operands per typical ALU instruction with examples of computers

Number of memory addresses	Maximum number of operands allowed	Type of architecture	Examples
0	3	Load-store	Alpha, ARM, MIPS, PowerPC, SPARC, SuperH, TM32, RISC-V
1	2	Register-memory	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memory-memory	VAX (also has three-operand formats)
3	3	Memory-memory	VAX (also has two-operand formats)

Classifying ISAs

Load-store (0, 3)

Advantages	Disadvantages
1. Simple, fixed-length instruction encoding. 2. Simple code generation model 3. Instructions take similar numbers of clocks to execute	1. Higher instruction count than architectures with memory references in instructions. 2. More instructions and lower instruction density lead to larger programs.

Classifying ISAs

Register-Memory (1, 2)

Advantages	Disadvantages
1. Data can be accessed without a separate load instruction first. 2. Instruction format tends to be easy to encode and yields good density.	1. Operands are not equivalent since a source operand in a binary operation is destroyed. 2. Encoding a register number and a memory address in each instruction may restrict the number of registers. 3. Clocks per instruction vary by operand location.

Classifying ISAs

Memory-Memory (2, 2) or (3,3)

Advantages	Disadvantages
1. Most compact. Doesn't waste registers for temporaries.	1. Large variation in instruction size, especially for three-operand instructions. 2. In addition, large variation in work per instruction. 3. Memory accesses create memory bottleneck.

CISC and RISC Architectures

- Typically, the design of a processor can be classified as
  - Complex Instruction Set Computing (CISC) architecture
  - Reduced Instruction Set Computing (RISC) architecture

- CISC based processors have the ability to execute complex operations using single or fewer instructions.

- RISC based processors only use simple instructions that can be executed within one clock cycle.

CISC and RISC Architectures

- Consider a line of code as

```
a = a  * b
```

  "a" and "b" are representing two numbers stored in memory

- CISC processor can complete this multiplication operation with one instruction:

```
MULT  a, b
```

  where MULT is as "complex instruction" that can load numbers from memory locations "a" and "b", multiply them and store the result back to memory location "a"

- As the code size is comparatively short, not much space is required to store instructions in the memory.

- The goal is to directly design hardware for such complex instructions.

CISC and RISC Architectures

- Same multiplication operation

```
a = a  * b
```

- RISC processor requires multiple instructions for this multiplication operation

```
lw  a1, a
lw  a2, b
mul a1, a1, a2
sw  a1, a
```

- Apparently there are more lines of code, more space is needed to store these instructions in memory.
- Requires more software (compiler) support to translate a high-level language code into this form.
- As the RISC can execute each instruction in only one clock cycle, the execution time of this code is almost the same as of a multi-cycle "MULT" command.

CISC and RISC Architectures

CISC	RISC
Primarily focused on hardware: hardware is complex whereas software is simple	Primarily focused on software: hardware is simple whereas software is complex
Multi-clock complex instructions: Perform complex operations using single/fewer instructions	Single-clock simple instructions: Requires multiple instructions to perform complex operations
There are no separate "LOAD" and "STORE“ instructions to access memory. The memory operations are fused within the core instructions [memory-to-memory]	There are separate "LOAD" and "STORE“ instructions to access memory. are independent instructions [Register-to-register]
Hardware required to decode instructions is complex	Hardware required to decode instructions is simple
Code size is small but more time is required to execute each instruction	Code size is large but requires less time to execute each instruction
May have variable length instructions	The instructions are of a fixed length

Latest trends

“Old” view of computer architecture

Instruction Set Architecture (ISA) design i.e. decisions regarding:

- Registers
- Memory addressing
- Addressing modes
- Instruction operands
- Available operations
- Control flow instructions
- Instruction encoding

Latest trends

“Latest” view of computer architecture

- Includes ISA, microarchitecture, hardware
- Instruction set architecture defines software access to the hardware
- Microarchitecture/organisation defines high-level aspects of a computer’s design
  - Such as the memory system, the memory interconnect, and the design of the processor
- Hardware refers to the specifics of a computer, including the detailed logic design and the packaging technology of the computer.
- Design goal is to maximize performance within constraints: cost, power, and availability

Latest trends

ARM: The Armv9 architecture is the latest generation Arm architecture supporting three ISAs: A64, A32 and T32.

- The A64 and A32 ISAs have fixed instruction lengths of 32-bits.
- The A64 ISA is a 64-bit architecture supported by the Armv9-A architecture whereas A32 is a 32-bit architecture supported by both A-profile and R-profile architectures.
- T32, formerly known as Thumb, supports 16-bit and 32-bit mixed-length instruction set.
- Custom Instructions for the Armv9-M architecture targets to improve performance and efficiency further by adding application domain specific features in small embedded processors

*https://developer.arm.com/architectures/instruction-sets

Latest trends

Intel: The IA-32 and Intel® 64 are the two ISAs from Intel

- IA-32 architecture is the instruction set architecture and programming environment for Intel's 32-bit microprocessors. IA-32 is based on the x86 ISA

- Intel® 64 architecture is the instruction set architecture and programming environment which is the superset of Intel's 32-bit and 64-bit architectures. It is compatible with the IA-32 architecture.

*https://software.intel.com/sites/default/files/managed/39/c5/325462-sdm-vol-1-2abcd-3abcd.pdf

Summary

- Instruction set architectures are classified based on the type of internal storage in a processor such as register, stack or accumulator.
- Register(load-store) architecture is used most widely due to its simplicity
- CISC is primarily focused on hardware: hardware is complex whereas software is simple
- RISC is primarily focused on software: hardware is simple whereas software is complex
- The word “computer architecture” covers all three aspects of computer design—instruction set architecture, organization or microarchitecture, and hardware